



Trabajo Práctico Final Sistemas Digitales 1



Generador de números aleatorios en un FPGA

Victor Curiel, Ximena Quenhan

Facultad de Ingeniería UNA, Ing. Mecatrónica

PALABRAS CLAVE

FPGA
LFSR
VHDL

RESUMEN

El presente el diseño e implementación de un generador de números aleatorios basado en un Registro de Desplazamiento de Retroalimentación Lineal (LFSR) en un FPGA. El generador utiliza un LFSR de tipo Galois, conocido por su eficiencia en la generación de secuencias pseudoaleatorias. Se detallan los procesos de implementación en VHDL y simulación, con un enfoque en la generación de números aleatorios a través de operaciones XOR. Finalmente, se presentan los resultados de simulación obtenidos mediante un test bench para verificar el correcto funcionamiento del sistema.

1. Introducción

En el contexto de los sistemas digitales y aplicaciones de criptografía, la generación de números aleatorios es fundamental para tareas como la encriptación de datos, simulaciones y pruebas de hardware. Uno de los métodos comúnmente utilizados es el Registro de Desplazamiento de Retroalimentación Lineal (LFSR), que permite generar secuencias pseudoaleatorias de manera eficiente mediante operaciones XOR. Este proyecto se enfoca en la implementación de un generador de números aleatorios en una FPGA, usando un LFSR de Galois. Se ha elegido la variante de Galois debido a su menor complejidad en la retroalimentación local. La implementación en VHDL permite realizar un diseño escalable y adaptable a distintas configuraciones, mientras que la simulación ayuda a verificar el comportamiento correcto del sistema.

2. Marco teorico.

2.1. LFSR.

Un Registro de Desplazamiento de Retroalimentación Lineal (Linear Feedback Shift Register) es un tipo de registro de desplazamiento utilizado comunmente en cifrados en flujo para la generación de secuencias pseudoaleatorias, mediante una simple operación XOR. El LFSR se configura inicialmente con un estado inicial, llamado semilla, que es una funcion lineal de su estado anterior.

En cada ciclo de reloj, los bits almacenados en las celdas del LFSR se desplazan y se generan nuevos bits pseudoaleatorios en función del resultado de la XOR de las diferentes posiciones.

Los LFSR cuentan con otras aplicaciones además de la generación de numeros pseudoaleatorios como los códigos detectores de errores, criptografía y pruebas de hardware.

2.2. LFSR de Galois.

Existen dos variantes para un LFSR: Galois y Fibonacci (2). En este caso, debido a su simplicidad y eficiencia elegimos trabajar con la variante de Galois. En el LFSR de Galois, en cada ciclo de reloj, el contenido de los registros se desplaza hacia la derecha, al igual que en el LFSR de Fibonacci. Sin embargo, la retroalimentación XOR se realiza localmente en algunos de los registros en lugar de recalcular la retroalimentación globalmente. Esto significa que la retroalimentación afecta a solo algunos bits intermedios en lugar de aplicarse únicamente en el bit más significativo.

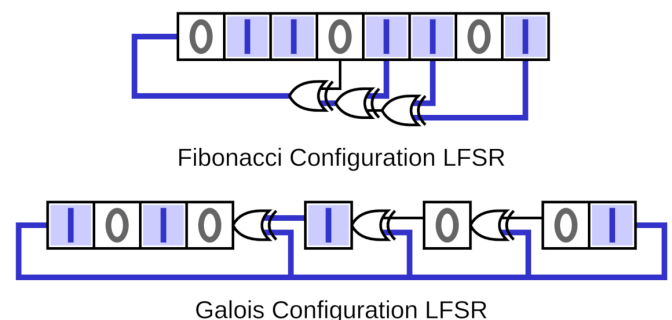


Fig. 1: (a) Configuración de Fibonacci. (b) Configuración de Galois. Fuente: <https://www.moria.us/articles/demystifying-the-lfsr/>

3. Diseño del proyecto.

3.1. Diagrama ASM.

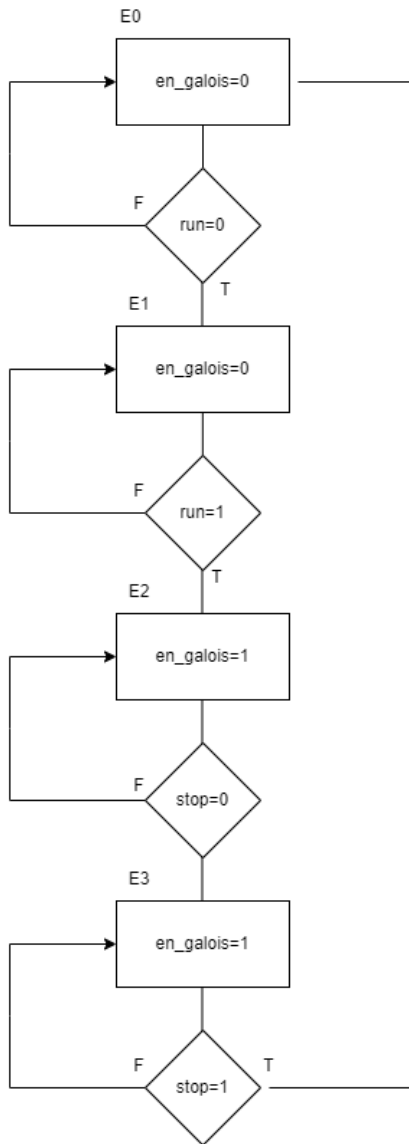


Fig. 2. Representación ASM del trabajo.

3.2. Implementación en VHDL.

```

1
2 library IEEE;
3 use IEEE.STD_LOGIC_1164.ALL;
4 use IEEE.NUMERIC_STD.ALL;
5
6 entity dagodigital_module is
7     Port ( clk,reset,run,stop : in  STD_LOGIC;
8           dot: out std_logic;
9           s_display : out  STD_LOGIC_VECTOR (6 downto 0)**;
10          dip : out  STD_LOGIC_VECTOR (2 downto 0));
11 end dagodigital_module;
12
13 architecture Behavioral of dagodigital_module is
14     type estado_type is (e0,e1,e2,e3);
15     signal estado, estado_sig: estado_type;
16     --Senhales de Galois
17     signal r,r_sig,aux: std_logic_vector
18         (15 downto 0);
19     signal aux2:std_logic_vector(2 downto 0);
20     signal aux3: std_logic_vector(6 downto 0);
21     signal en_galois,en: std_logic;
22     signal c,c_sig:unsigned(27 downto 0);
23 begin
    
```

```

24 process(clk) --Proceso de sincronizacion
25 begin
26     if(clk'event and clk='1') then
27         if reset='0' then
28             estado<=e0;
29         else
30             estado<=estado_sig;
31         end if;
32     end if;
33 end process;
34
35 --Controla la transicion de estados
36 --(ESTADO SIGUIENTE DECODE)
37 process(estado,run,stop)
38 begin
39     estado_sig<=estado;
40     case(estado) is
41         when e0 =>
42             if run='1' then
43                 estado_sig<=e1;
44             end if;
45         when e1=>
46             if run='0' then
47                 estado_sig<=e2;
48             end if;
49         when e2=>
50             if stop='1' then
51                 estado_sig<=e3;
52             end if;
53         when e3=>
54             if stop='0' then
55                 estado_sig<=e0;
56             end if;
57         when others =>
58             estado_sig<=e0;
59     end case;
60 end process;
61
62 --Controla en_galois que se utiliza en
63 --la logica de Galois (SALIDA DECODE)
64 process(estado,run,stop)
65 begin
66     en_galois<='0';
67     case (estado) is
68         when e0 =>
69             en_galois<='0';
70         when e1 =>
71             if run='0' then
72                 en_galois<='1';
73             else
74                 en_galois<='0';
75             end if;
76         when e2 =>
77             en_galois<='1';
78         when e3 =>
79             if stop<='0' then
80                 en_galois<='0';
81             else
82                 en_galois<='1';
83             end if;
84         when others =>
85             en_galois<='0';
86     end case;
87 end process;
88
89 process(clk)
90 begin
91     if (clk'event and clk='1') then
92         if(reset='0') then
93             r<="1010110011100001";
94         else
95             r<=r_sig;
96         end if;
97     end if;
98 end process;
99
100 process(clk) --Divisor de frecuencia
101 begin
102
    
```

```

103         if clk'event and clk='1' then
104             if reset='0' then
105                 c<=(others =>'0');
106             else
107                 c<=c_sig;
108             end if;
109         end if;
110     end process;
111
112     c_sig <= (others=> '0') when c=15000000
113 else c+1;
114     en<='1' when c=15000000 else '0';
115
116     --Algoritmo de Galois
117     aux<=r(0)&r(15 downto 1);
118     r_sig<=
119         aux(15) & aux(14) & (aux(0) XOR aux(14)) &
120         (aux(0) XOR aux(13)) &
121         aux(11) & (aux(0) XOR aux(11)) &
122         aux(9 downto 0) when (en_galois='1') and
123         en='1' else r;
124
125     aux2<=r(15 downto 13);
126
127     --Mostrar en el display
128     with aux2 select
129         aux3<="0110000" when "000",
130             "0110000" when "001",
131             "1101101" when "010",
132             "1111001" when "011",
133             "0110011" when "100",
134             "1011011" when "101",
135             "1011111" when others;
136
137     --Negamos porque el display es
138     --catodo comun
139     s_display<=not(aux3);
140     dip<="011";
141     dot<='1';
142
143 end Behavioral;

```

3.3. Simulación.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY dadodigital_simulacion IS
5  END dadodigital_simulacion;
6
7  ARCHITECTURE behavior OF dadodigital_simulacion IS
8
9      COMPONENT dagodigital_module
10     PORT(
11         clk : IN  std_logic;
12         reset : IN  std_logic;
13         run : IN  std_logic;
14         stop : IN  std_logic;
15         s_display : OUT  std_logic_vector
16         (6 downto 0);
17         dip : OUT  std_logic_vector(2 downto 0)
18     );
19     END COMPONENT;
20
21
22     --Inputs
23     signal clk : std_logic := '0';
24     signal reset : std_logic := '0';
25     signal run : std_logic := '0';
26     signal stop : std_logic := '0';
27
28     --Outputs
29     signal s_display : std_logic_vector
30     (6 downto 0);
31     signal dip : std_logic_vector(2 downto 0);
32
33     -- Clock period definitions
34     constant clk_period : time := 10 ns;
35

```

```

36 BEGIN
37
38     -- Instantiate the Unit Under Test (UUT)
39     uut: dagodigital_module PORT MAP (
40         clk => clk,
41         reset => reset,
42         run => run,
43         stop => stop,
44         s_display => s_display,
45         dip => dip
46     );
47
48     -- Clock process definitions
49     clk_process :process
50     begin
51         clk <= '0';
52         wait for clk_period/2;
53         clk <= '1';
54         wait for clk_period/2;
55     end process;
56
57     -- Stimulus process
58     stim_proc: process
59     begin
60         wait for 100 ns ;
61         reset <= '0';
62
63         wait for 100 ns ;
64         reset <= '1';
65
66         wait for 100 ns;
67         run<='1';
68
69         wait for 100 ns;
70         run<='0';
71
72         wait for 100 ns;
73         stop<='1';
74
75         wait for 100 ns;
76         stop<='0';
77
78         wait for 50 ns;
79         run<='1';
80
81         wait for 50 ns;
82         run<='0';
83
84         wait for 50 ns;
85         stop<='1';
86
87         wait for 50 ns;
88         stop<='0';
89
90         wait ;
91     end process;
92
93 END;

```

4. Resultados.

Utilizando los códigos mostrados en los incisos anteriores se obtuvieron los siguientes resultados mediante el test bench:

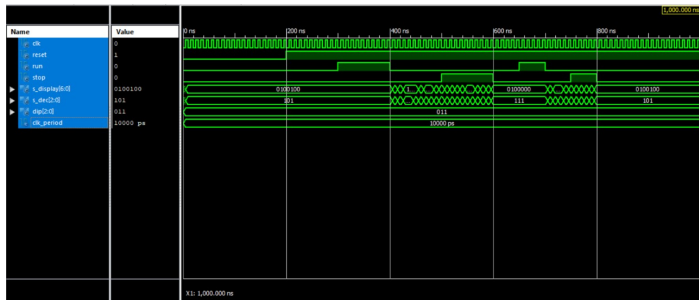


Fig. 3. Simulación mediante el test bench.

En la simulación se puede ver como se generan correctamente los números pseudoaleatorios mediante la activación del botón *run* seguido de la activación del botón *stop*.

Y por último se realizó la implementación final en el FPGA obteniendo los resultados esperados como se puede ver en las figuras 4, 5 y 6, en las que se muestran el número pseudoaleatorio obtenido del dado digital.

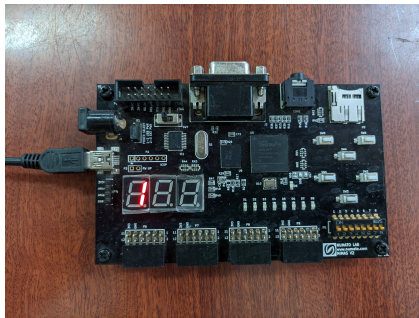


Fig. 4. Respuesta 1 del dado digital.

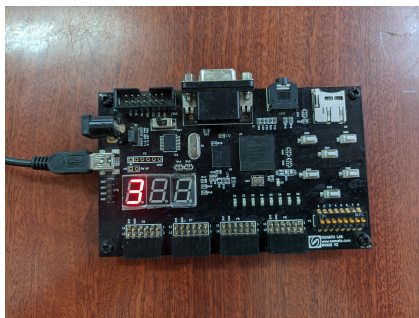


Fig. 5. Respuesta 3 del dado digital.

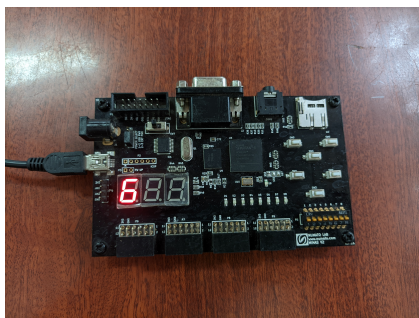


Fig. 6. Respuesta 6 del dado digital.

5. Conclusión.

El generador de números aleatorios basado en LFSR de Galois implementado en FPGA ha demostrado ser efectivo para la generación de secuencias pseudoaleatorias, necesarias en variadas aplicaciones. La simulación realizada a través de un test bench confirmó que la estructura del LFSR permite obtener resultados predecibles y reproducibles con baja complejidad de diseño. Esta implementación en VHDL es escalable y adaptable, lo que abre la posibilidad de ampliaciones para sistemas con mayores requerimientos de seguridad o precisión en la generación de números aleatorios.

6. Referencias.

- [1] M. Fukar, *LFSR*, GitHub repository, 2020. Available: <https://github.com/mfukar/lfsr>.
- [2] Texas Instruments, *LFSR Analysis*, Application Note SCTA036A, 2003. Available: <https://www.ti.com/lit/an/scta036a/scta036a.pdf>.
- [3] Galois, *Galois LFSR API Documentation*, Galois Documentation, version 0.0.24. Available: <https://galois.readthedocs.io/en/v0.0.24/api/galois.LFSR.html>.
- [4] A. L. H. W. Moria, *Demystifying the LFSR*, 2020. Available: <https://www.moria.us/articles/demystifying-the-lfsr/>.